DynLang-SLAM: How Our Pipeline Differs from Existing 3DGS SLAM Methods

Ankur Guruprasad
EEE 515 – Computer Vision, Arizona State University
agurupr6@asu.edu

April 2026

1 Introduction

DynLang-SLAM is a 3D Gaussian Splatting (3DGS) SLAM system that combines real-time camera tracking with open-vocabulary language understanding and dynamic object handling. This report compares our architectural choices against existing 3DGS SLAM papers — SplaTAM [**?**], MonoGS [**?**], Gaussian-SLAM [**?**], GS-SLAM [**?**], and RTG-SLAM [**?**] — highlighting where we follow established patterns, where we innovate, and why.

2 System Architecture Comparison

2.1 Overall Pipeline

All 3DGS SLAM systems share the same high-level loop: *Track* (optimize camera pose) → *Keyframe check* → *Map* (optimize Gaussians). The key differences lie in the details of each step.

3 Tracking Innovations

3.1 Literature-Informed Loss Schedule

What others do: Every existing 3DGS SLAM system uses **constant loss weights** during tracking optimization. SplaTAM uses depth=1.0, RGB=0.5 (depth is 2× RGB). Gaussian-SLAM uses equal weights. No paper modulates loss weights across iterations.

What we do: DynLang-SLAM uses a **rising photometric schedule** — the first literature-aware dynamic loss weighting in 3DGS SLAM:

$$w_{\text{depth}}(t) = 1.0 \cdot w_d \quad \text{(constant)} \tag{1}$$

$$w_{\text{rgb}}(t) = (0.5 + 0.5t) \cdot w_r \quad t \in [0, 1] \tag{2}$$

$$w_{\text{ssim}}(t) = (0.5 + 0.5t) \cdot w_s \tag{3}$$

where $w_d = 1.2$, $w_r = 0.5$, $w_s = 0.2$ are the base weights, and t is the normalized iteration progress.

Why: Early iterations need a strong geometric anchor (depth) to pull the pose into the correct basin of attraction. Late iterations benefit from photometric detail (RGB, SSIM) for sub-pixel refinement. Critically, depth is *never reduced* below baseline — our literature survey found that reducing depth weight causes long-term drift (validated by our own room0 regression experiment).

3.2 Constant-Velocity Pose Initialization

What others do: Most methods initialize the current pose estimate as the previous frame's pose (identity motion model). RTG-SLAM uses ICP which implicitly provides better initialization.

What we do: We extrapolate using the two most recent poses:

$$\hat{T}_t = T_{t-1} \cdot (T_{t-2}^{-1} \cdot T_{t-1}) \tag{4}$$

This constant-velocity model provides better initialization for scenes with consistent camera motion (e.g., room1 with its sweeping trajectory).

Why: With only 50 tracking iterations, the optimizer has limited budget. Starting closer to the true pose means convergence is faster and more reliable, especially for scenes with non-trivial camera motion.

3.3 Quaternion Normalization per Step

What others do: Most methods optimize quaternions freely and normalize only at the final output.

What we do: After every optimizer step, we normalize the quaternion to unit length:

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|} \tag{5}$$

Why: Adam's adaptive learning rate scales each quaternion component independently. Without normalization, the magnitude drifts, causing the effective rotation step size to vary unpredictably. Per-step normalization keeps the optimization on the unit quaternion manifold.

Table 1: Architectural comparison of 3DGS SLAM methods. Blue = DynLang-SLAM specifics.

Feature	**SplaTAM**	**MonoGS**	**GS-SLAM**	**G-SLAM**	**RTG-SLAM**	**Ours**
Renderer	diff-gaussian	custom	diff-gaussian	gsplat	custom	gsplat
Tracking method	Loss opt.	Direct opt.	Loss opt.	Loss opt.	ICP	Loss opt.
Depth loss	L1	L1	L1	L1	N/A (ICP)	L1
RGB loss	L1	L1	L1 + SSIM	L1	L1	L1 + SSIM
Depth weight	1.0	—	0.8	1.0	—	1.2
RGB weight	0.5	—	0.8	1.0	—	0.5
Loss scheduling	None	None	None	None	None	Rising RGB
Tracking iters	40	—	—	—	—	50
LR scheduling	None	—	Coarse LR	None	—	Cosine (map)
Pose init	Previous	Previous	Previous	Previous	ICP	Const. velocity
Densification	SfM-style	—	Coarse-fine	None	Tile-based	Grad + render
Dynamic objects	No	No	No	No	No	Planned (YOLO)
Language	No	No	No	No	No	Planned (CLIP)

3.4 Motion Magnitude Clamping

What others do: No explicit motion clamping in any published method.

What we do: If the estimated translation exceeds 5× the average recent motion (or 5cm minimum), we clamp it to that bound while preserving direction.

Why: Single-frame drift from tracking failure can compound catastrophically. This is a safety net, not a correction — it fires rarely but prevents catastrophic jumps.

4 Mapping Innovations

4.1 Gradient + Rendering-Guided Densification

What others do: SplaTAM uses silhouette-based expansion only. Original 3DGS uses position gradient thresholding. RTG-SLAM uses tile-based heuristics.

What we do: We combine both approaches:

1. **Gradient-based**: Split large / clone small Gaussians where position gradient $\|\nabla\mu\| > \tau$
2. **Rendering-guided**: When color error is high (> 0.3) or coverage is low ($\alpha < 0.5$), we halve the gradient threshold τ to densify more aggressively in under-reconstructed regions
3. **Silhouette expansion**: Standard SplaTAM-style addition of new Gaussians where $\alpha < 0.5$

Why: Silhouette expansion alone misses regions where Gaussians exist but are too large/blurry (they contribute to alpha but not to accurate rendering). Gradient-based densification catches these cases. The rendering-guided threshold adaptation is novel — it bridges the gap between the two signals.

4.2 Isotropic Regularization

What others do: MonoGS proposes isotropic regularization. Most other methods do not use it.

What we do: We penalize elongated Gaussians:

$$\mathcal{L}_{\text{iso}} = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{3}|s_{ij} - \bar{s}_i| \tag{6}$$

where s_{ij} are the activated scale components and $\bar{s}_i$ is their mean.

Why: In SLAM, elongated Gaussians create ambiguous depth and can "slide" during tracking. Isotropic Gaussians provide more reliable geometric primitives.

4.3 Cosine Annealing LR in Mapping

What others do: Constant LR (SplaTAM, Gaussian-SLAM), or coarse-to-fine LR (GS-SLAM uses higher LR for first 5 iterations).

What we do: CosineAnnealingLR from full LR to 0.3× LR over 60 mapping iterations. *Note:* We do NOT use LR scheduling in the tracker (50 iterations is too short; LR decay traps the pose in local minima).

Why: Mapping iterations (60) are longer and benefit from aggressive early optimization followed by fine-grained convergence. Tracking (50 iterations) needs consistent step sizes to escape local minima.

4.4 Fast SE(3) Inverse

What others do: Standard `torch.inverse()` for pose inversion.

What we do: Exploit the rigid-body structure of SE(3):

$$T^{-1} = \begin{bmatrix} R^T & -R^T\mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \tag{7}$$

Why: Avoids the $O(n^3)$ general matrix inversion. Mathematically identical for valid SE(3) matrices, faster in practice.

5 Planned: Language Pipeline

The key differentiator of DynLang-SLAM is the integration of open-vocabulary language features into the Gaussian map, enabling 3D text-based object queries. No existing 3DGS SLAM system has this capability.

Architecture (LangSplat-inspired):

1. SAM mask generation at 3 semantic scales (subpart, part, whole)
2. Per-mask CLIP encoding (ViT-L/14, 768-dim)
3. Online autoencoder compression (768 → 16-dim latent)
4. Fusion into Gaussian `lang_feats` via differentiable splatting
5. Open-vocabulary 3D query via cosine similarity

Key challenge: LangSplat trains its autoencoder offline on all images. In SLAM, we must train it *online* as frames arrive — requiring a warmup phase and careful generalization.

6 Planned: Dynamic Object Masking

Another unique capability: dynamic object detection and exclusion from the static map.

Architecture:

1. YOLOv8 instance segmentation per frame
2. Temporal consistency filtering (min 2 detections in 3-frame window)
3. Dynamic mask applied to both tracking and mapping losses
4. Contamination cleanup: prune Gaussians that were initialized from dynamic regions

No existing 3DGS SLAM paper handles dynamic objects at the framework level.

7 Summary of Novelties

8 What We Tried and Rejected

Importantly, our design is informed by systematic ablation:

- **Coarse-to-fine tracking** (GS-SLAM): Regressed room0 from 1.43→4.07cm. Indoor scenes need full-resolution detail.
- **LR scheduling in tracker**: Regressed room0 to 7.03cm. 50 iterations is too short for LR decay.

Table 2: Novel contributions vs. existing methods.

Contribution	**Status**
Rising photometric loss schedule	Implemented
Constant-velocity pose init	Implemented
Per-step quaternion normalization	Implemented
Motion magnitude clamping	Implemented
Gradient + rendering densification	Implemented
Isotropic regularization	Implemented
Cosine LR in mapping (not tracking)	Implemented
Fast SE(3) inverse	Implemented
Language-embedded Gaussians (CLIP)	Planned (Week 2)
Dynamic masking (YOLOv8)	Planned (Week 3)

- **Huber depth loss**: Hurt convergence. Large depth errors are the signal pulling wrong poses back — clipping them is counterproductive.
- **Depth-aware scale initialization**: 30% fewer Gaussians (69K vs 99K). Larger initial scales meant fewer "unmapped" pixels, reducing map coverage.
- **Reducing depth weight below baseline**: Caused room0 to regress at 2000 frames (3.5cm vs 2.0cm baseline). Literature confirmed: depth must remain the geometric anchor.

References

[1] N. Keetha et al., "SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM," CVPR 2024.

[2] H. Matsuki et al., "Gaussian Splatting SLAM," CVPR 2024 Highlight.

[3] V. Yugay et al., "Gaussian-SLAM: Photo-realistic Dense SLAM with Gaussian Splatting," 2024.

[4] C. Yan et al., "GS-SLAM: Dense Visual SLAM with 3D Gaussian Splatting," CVPR 2024.

[5] Z. Peng et al., "RTG-SLAM: Real-time 3D Reconstruction at Scale using Gaussian Splatting," SIGGRAPH 2024.

[6] M. Qin et al., "LangSplat: 3D Language Gaussian Splatting," CVPR 2024 Highlight.

[7] J. Shi et al., "Language Embedded 3D Gaussians for Open-Vocabulary Scene Understanding," CVPR 2024.